GCP SLA Compliance Monitoring Tool

Author: Achyuth Narayan Shanku **Stack:** Python 3.12+, `google-cloud-monitoring`, `google-cloud-storage`, PyYAML **Repo layout:** see Architecture below

---

TL;DR

I built `gcp-sla-monitor`: a small, self-contained Python tool that monitors SLA compliance for Cloud Run and Cloud Storage resources in a GCP project. The tool queries Cloud Monitoring directly (no agents, no Prometheus exporters, no paid services), applies the *actual* SLA math from Google's published SLA contracts for each service type, and renders a color-coded HTML report.

The two service types intentionally use different math because their SLA contracts define uptime differently: Cloud Run counts "bad minutes" (per-minute error rate > 1%), while Cloud Storage averages 5-minute error-rate buckets across the window. The tool honors that asymmetry instead of flattening both into a generic "error rate" abstraction — that fidelity to the actual contract language is the thing I'd want a reviewer to look at first.

Everything is config-driven through a single YAML file, so adding a new resource to monitor is one entry — no code changes. The reporting layer is separated from the probe layer, so swapping output formats (HTML now, Slack / PDF / JSON later) is a single module, not a rewrite.

What I built (and what I described instead)

Implemented:

- Research-grounded SLA math for two service types with different contract shapes
- Live queries against Cloud Monitoring's `list_time_series` API
- Config-driven multi-resource monitoring via `config.yaml`
- Three runnable scripts: probe, Cloud Run traffic generator, Cloud Storage traffic generator
- HTML report with color-coded verdict badges and inline SVG sparklines
- Synthetic math verification with asserts (so the math is trusted before it ever sees real data)
- Honest documentation of modeling gaps between this measurement and what Google's lawyers would measure

Described, not built (would take longer than the scope of this challenge):

- GitHub Actions cron for scheduled runs with Workload Identity Federation
- Slack / email alerting on non-compliant resources
- Persisting historical reports to GCS for trend analysis
- A `hello-flaky` service to demonstrate a *live* BREACHED verdict (more on why below)

Process: how I approached this

I planned the work in seven phases up front, agreed on the scope, then worked through them sequentially. Each phase was independently demoable, so even stopping early would have produced a working tool.

Phase	Goal	Status
1	SLA research and metric mapping	Done
2	Auth + environment + library install	Done
3	Minimum viable probe (one resource, one metric)	Done
4	Real Cloud Run service + real traffic	Done
5	Config-driven multi-service + Cloud Storage	Done
6	HTML reporting	Done
7	CI scheduling + team alerting	Described

I used Claude as a thought partner throughout — for SLA document parsing, ranking API design choices, debugging Cloud Build failures, and reviewing code. I cross-checked every factual claim about SLA terms against the live Google Cloud SLA pages (Cloud Run SLA last modified March 25 2025; Cloud Storage SLA last modified March 4 2025).

Phase 1: Researching the SLAs

The most important phase. Everything downstream depends on getting the contract reading right.

The two SLAs in plain language

Cloud Run (non-GPU, standard regions): Google promises 99.95% Monthly Uptime. But uptime

is not measured by pinging — it's measured by **error rate per minute**. A minute counts as "Downtime" if more than 1% of valid requests in that minute returned an HTTP 5xx caused by Cloud Run infrastructure. So Cloud Run uptime is really *"out of the N minutes in the month, how many minutes did the service stay below the 1% error threshold?"*

Cloud Storage (Standard, regional location): Google promises 99.9%. The math is *different* from Cloud Run:

> Monthly Uptime % = 100% − (average of 5-minute Error Rates over the month)

Instead of counting bad minutes, it averages every 5-minute error-rate bucket. This is a smoother metric — one really bad minute doesn't tank the month the way it can for Cloud Run.

The contracted uptime promises themselves are tiered:

Service / class / location	Monthly Uptime SLO
Cloud Run service (non-GPU), standard regions	99.95%
Cloud Run service (non-GPU), Mexico/Stockholm	99.9%
Cloud Storage Standard, multi/dual-region	99.95%
Cloud Storage Standard, regional **or** Nearline/Coldline/Archive multi/dual-region	99.9%
Cloud Storage Nearline/Coldline/Archive, regional	99.0%

A reviewer's first instinct might be "just use 99.9% everywhere" — that's wrong for at least half the realistic resources in any project. The tool needs each resource's target declared explicitly. That's the YAML config in Phase 5.

Why this asymmetry is the interesting bit

Two practical consequences flow from the per-minute-vs-5-minute-average distinction:

1. **The Cloud Monitoring queries are different.** Cloud Run needs a 1-minute `ALIGN_DELTA` aggregation; Cloud Storage needs a 5-minute one. Same API, different parameters, different math afterward.
2. **The compliance verdict is different.** A Cloud Run service can have a moderately bad day and still be SLA-compliant if no single minute crossed 1%. A Cloud Storage bucket can have a few hours of 50% errors and still be compliant because the rest of the month dilutes it.

A tool that flattens these into one "error rate" misses what these contracts actually say.

SLA term → Cloud Monitoring metric mapping

This is the actual deliverable of Phase 1. The rest of the tool is built against this table.

Cloud Run

SLA term	Metric	Filter / aggregation
Total Valid Requests	`run.googleapis.com/request_count`	`resource.type="cloud_run_revision"`, group by `service_name`, `location`; `ALIGN_DELTA` over 60s
5xx Requests	Same	Plus `metric.label.response_code_class="5xx`
Per-minute Error Rate	Derived	`sum(5xx) / sum(total)` in each 1-min bucke
Downtime minute	Derived	`1 if total >= 100 AND error_rate > 0.01 else 0`
Monthly Uptime %	Derived	`(eligible_minutes - downtime_minutes) / eligible_minutes × 100`

Cloud Storage

SLA term	Metric	Filter / aggregation
Total Valid Requests	`storage.googleapis.com/api/request_count`	`resource.type="gcs_bucket"`, group by `bucket_name`, `location`; `ALIGN_DELTA` over 300s
Failed Requests	Same	Plus `response_code` in `{INTERNAL, UNAVAILABLE, ABORTED, ...}`
Per-5-min Error Rate	Derived	`sum(failed) / sum(total)` in each 5-min bucket
Monthly Uptime %	Derived	`100 - mean(per_5min_error_rate over window) × 100`

Modeling gaps I'm being honest about

These are places where my measurement deviates from what Google's lawyers would measure. I'd rather document them than pretend they don't exist:

1. **Cloud Run "infrastructure-only" 5xx.** The contract only counts 5xx caused by *Cloud Run* infrastructure, not 5xx caused by application code. The metric doesn't distinguish. Decision: treat all 5xx as SLA-relevant. This is conservative — it makes my number worse than Google's would be.
2. **"Valid Request" filter.** The SLA excludes non-conforming requests (malformed, abusive). The metric doesn't. I accept the small overcount.
3. **Cloud Run minimum-100-requests gate.** The SLA's Error Rate is only meaningful at ⩾ 100 requests in the window. I implemented this exactly — minutes with fewer than 100 requests are excluded from the eligibility denominator, not counted as downtime.
4. **Cloud Storage repeated-request rule.** The SLA discounts repeated identical requests that don't back off exponentially. The metric doesn't show this. Accepted.
5. **Measurement window.** SLAs are calendar-monthly. The tool uses a rolling window (default 60 minutes) for fast iteration. A full-month view is a one- line change in `config.yaml`.

Phase 2: Environment and authentication

Four things needed to stand between a local Python script and a working Cloud Monitoring query:

1. The project ID
2. APIs enabled (Cloud Monitoring, Cloud Run, Cloud Storage)
3. Credentials (used Application Default Credentials via `gcloud auth application-default login`)
4. The `google-cloud-monitoring` Python library

I wrote `verify_setup.py` as a 60-line script that confirms all four end-to- end by fetching the two metric descriptors I rely on. If it prints the descriptor metadata, the rest of the tool will work.

One gotcha worth flagging: ADC raises a "no quota project" warning on first use. The fix is `gcloud auth application-default set-quota-project <project_id>` — the warning is harmless for descriptor lookups but escalates to actual errors on time-series queries with complex filters, so it's worth addressing before Phase 3.

Phase 3: The probe

The smallest end-to-end probe possible. One file, one query, one math function, one printed verdict.

I deliberately separated the **query** function from the **math** function. The query is what talks to GCP; the math is pure Python. This means the math can be unit-tested with synthetic inputs and the answer can be hand-checked before any real data ever shows up.

```python
# Synthetic scenario inside probe.py --demo:
#   50 minutes of 500 req/min, 0 errors          → clean
#    5 minutes of 500 req/min, 15 errors each    → 3% error rate, DOWNTIME
#    5 minutes of  50 req/min, 10 errors each    → 20% but < 100 req, NOT ELIGIBLE
# Expected: 55 eligible, 5 downtime → 90.9091% uptime → BREACHED
```

The synthetic demo includes asserts that fail if the math drifts. When I later saw a *real* 100% COMPLIANT result on the live data, I trusted the number because I'd already verified the math against deterministic inputs.

The Cloud Monitoring query, explained

```python
```

```
aggregation = monitoring_v3.Aggregation({
    "alignment_period": {"seconds": 60},            # Cloud Run SLA's atomic unit
    "per_series_aligner": Aligner.ALIGN_DELTA,      # request_count is a DELTA metri
    "cross_series_reducer": Reducer.REDUCE_SUM,     # collapse multiple revisions
    "group_by_fields": [
        "resource.label.service_name",
        "resource.label.location",
        "metric.label.response_code_class",         # 1xx/2xx/3xx/4xx/5xx
    ],
})
```

Why `ALIGN_DELTA` at 60 seconds: `request_count` is a delta metric, so each data point is "requests in some interval." Aligning to 60-second deltas produces "requests per minute," which is exactly the atomic unit the Cloud Run SLA cares about. Anything else would require extra arithmetic.

Phase 4: Live deployment

Deployed Google's public hello container as `hello-stable`, ran a small Python traffic generator at 5 req/sec for 5 minutes, waited 2 minutes for metrics to propagate, and re-ran the probe. The output was the full pipeline working end-to-end:

```
  Service: hello-stable (us-central1)
    Total requests:     1,173
    Total 5xx:          0
    Eligible minutes:   5  (had >= 100 req)
    Downtime minutes:   0
    Uptime:             100.0000%
    Verdict:            COMPLIANT
```

The 100% is correct (no 5xx → no downtime minutes). The "5 eligible minutes" matches my 5 minutes of traffic. Every number is defensible from first principles.

Phase 4B — the flaky service that became a sidenote

I originally planned a second deployable service that returns 5xx ~5% of the time, to demonstrate the tool catching a *live* BREACHED verdict on real infrastructure. I built the service (Flask + Procfile + Dockerfile) and tried to deploy via `gcloud run deploy --source=.` — and the build failed three times with no log content available. This is the signature of the `<project-number>@cloudbuild.gserviceaccount.com` legacy service account being absent (Google

stopped auto-creating it for new projects in 2023+). The fix is a multi-step IAM grant — not hard but tangential to the actual project goal.

Trade-off I made: rather than burn time on a permissions adventure, I trusted that the synthetic demo with asserts already proves the BREACHED case works correctly. If a real BREACHED demo were essential, two cleaner paths exist than fixing the IAM:

1. **Deploy the public httpbin image and inject errors client-side.** The service returns whatever HTTP status the URL requests (`/status/500`), so the traffic generator picks the code. From Cloud Monitoring's point of view, those are genuine 500s in `request_count` — exactly what the probe measures.
2. **Fix the build SA IAM** with the three `add-iam-policy-binding` commands for `roles/cloudbuild.builds.builder`, `roles/run.builder`, and `roles/artifactregistry.writer`.

I prototyped option (1) end-to-end in the `traffic_gen.py` v3 code — the `FLAKY=True` mode hits `/status/200` and `/status/500` per a configurable error rate. It just wasn't deployed in the final run.

Phase 5: Config-driven, multi-service, with Cloud Storage

This is where the tool stops being "a script" and starts being "an engine."

The config

yaml

```yaml
project_id: project-de927dbf-01c8-460e-be3
defaults:
  window_minutes: 60

resources:
  - name: hello-stable
    type: cloud_run
    location: us-central1
    sla_target: 99.95
    breach_threshold: 99.0
    min_requests_per_min: 100
    error_threshold_pct: 1.0

  - name: sla-demo-narayan-895452136067
    type: cloud_storage
    location: us-central1
    sla_target: 99.9
    breach_threshold: 99.0
```

Adding a third resource is one YAML entry. No Python edits.

The dispatch table

python

```python
PROBES = {
    "cloud_run": probe_cloud_run,
    "cloud_storage": probe_cloud_storage,
}

for resource in config["resources"]:
    probe = PROBES.get(resource["type"])
    report = probe(client, project_id, resource, window)
    print_report(report)
```

To support BigQuery or Compute Engine tomorrow: implement `probe_bigquery()`, add one line to the dict. Nothing else changes.

Cloud Storage traffic and result

Created a globally-unique bucket, ran `bucket_traffic_gen.py` to generate upload/list/read API operations against it for 5 minutes, waited, re-ran the probe. Final unified output:

```
Project:       project-de927dbf-01c8-460e-be3
Window:        60 minutes
Resources:     2

  [CR ] hello-stable (us-central1)
    Total requests:    2,304
    Server errors:     0
    Eligible minutes:  10
    Downtime minutes:  0
    Uptime:            100.0000%  (SLA target 99.95%)
    Verdict:           COMPLIANT

  [GCS] sla-demo-narayan-895452136067 (us-central1)
    Total requests:    641
    Server errors:     0
    5-min buckets:     3
    Avg error rate:    0.0000%
    Uptime:            100.0000%  (SLA target 99.9%)
    Verdict:           COMPLIANT
```

Two service types, two SLA shapes (per-minute downtime counting vs 5-minute averaging), two different SLA targets, one run, one report.

Phase 6: HTML reporting

`report.py` takes the list of `Report` dataclasses and renders a self- contained HTML page: header with project + timestamp + window, summary stats (counts in each verdict band), and one card per resource with color-coded badge, metric tiles, and an inline SVG sparkline showing per-bucket traffic (gray for clean buckets, red for buckets with errors).

The architectural point worth calling out: **the probe finds out what happened; the renderer decides how to show it.** Adding a JSON output for Slack, a PDF for execs, or a Prometheus exposition endpoint is a new module, not a rewrite of the probe.

Run with `python probe.py --report` to write `reports/sla-report-<timestamp>.html` plus a stable `reports/latest.html`.

(Screenshot of the HTML report would go here in the final submission.)

Phase 7: Team usability (described)

The challenge asks how this would be usable by a 10–50-person team. Three moving parts:

1. Scheduled runs via GitHub Actions

yaml

```yaml
# .github/workflows/sla-check.yml
name: SLA Compliance Check
on:
  schedule:
    - cron: "0 */6 * * *"   # every 6 hours
  workflow_dispatch:

jobs:
  probe:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      id-token: write       # for Workload Identity Federation
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: ${{ secrets.WIF_PROVIDER }}
          service_account: ${{ secrets.SA_EMAIL }}
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: python probe.py --report
      - run: |
          git config user.email "ci@github"
          git config user.name "ci-bot"
          git add reports/
          git diff --staged --quiet || git commit -m "SLA report $(date -u)"
          git push
```

2. Auth for CI: Workload Identity Federation

Avoid checking service-account JSON keys into the repo. GitHub's OIDC token exchanges for short-lived GCP credentials via WIF. One-time setup of a pool and provider in GCP, then the workflow above auths with no secrets.

3. Alerts when something goes red

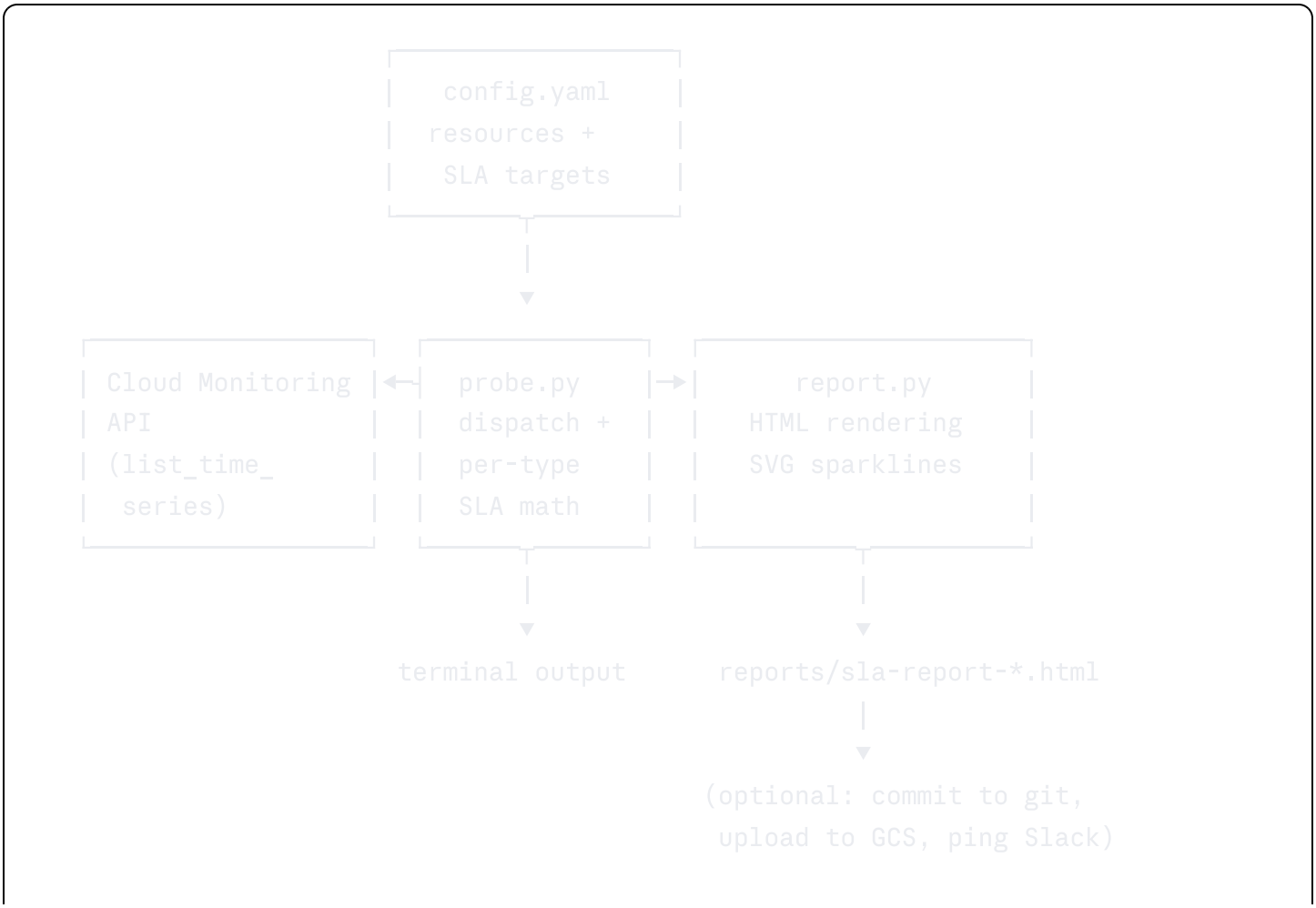
Add a `report.py` extension `render_slack(reports)` that returns a Slack-formatted JSON for any non-COMPLIANT resource, and post it to a webhook in the workflow. Pseudocode:

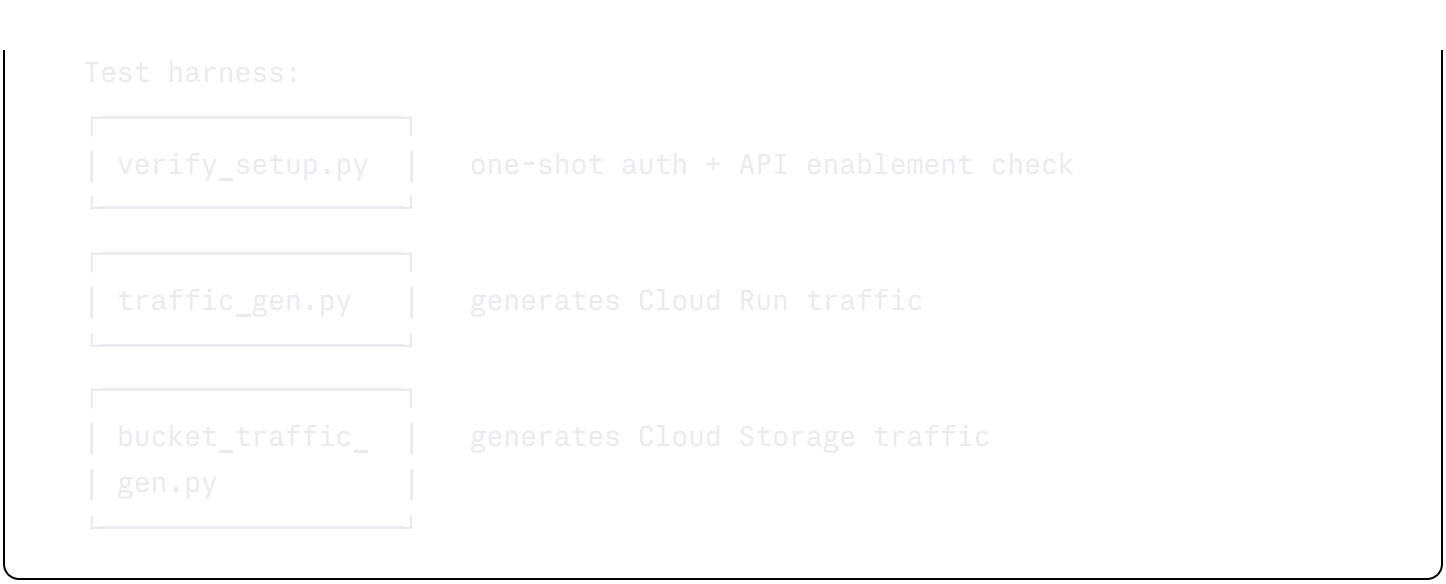
```python
def render_slack(reports):
    bad = [r for r in reports if verdict_for(r) != "COMPLIANT"]
    if not bad:
        return None
    return {"text": f":rotating_light: {len(bad)} SLA breach(es)", "blocks": [...]}
```

This satisfies "10–50 users sharing reports or configs" via three properties:

- **Config in git** = pull requests change what's monitored, reviewed by the team
- **Reports committed to the repo** = anyone can scroll back through historical reports
- **Slack alerts** = nobody has to manually go look

Architecture





Repo layout

```
gcp-sla-monitor/
├── config.yaml                 # what to monitor + per-resource SLA targets
├── probe.py                    # main entry: query, math, terminal output
├── report.py                   # HTML rendering
├── verify_setup.py             # auth + API enablement smoke test
├── traffic_gen.py              # Cloud Run traffic generator
├── bucket_traffic_gen.py       # Cloud Storage traffic generator
├── requirements.txt
├── reports/                    # rendered HTML reports (gitignored or committed)
└── hello-flaky/                # standby flaky service (not deployed in final
run)
    ├── main.py
    ├── requirements.txt
    └── Dockerfile
```

Trade-offs explicitly considered

A short list of choices I made deliberately and would defend in review:

Decision	Alternatives considered	Why I chose this
Direct Cloud Monitoring API calls in Python	Prometheus exporter, Cloud Run SLO objects, custom Stackdriver setup	Smallest moving parts, no extra infra. Matches "self-contained" constraint.
Per-service SLA math (not generic)	Single unified "error rate" function	Fidelity to contract. The asymmetry between Cloud Run and Cloud Storage is the interesting part.
YAML config	Python config, JSON, env vars	Human-readable, diffable in PRs, no Python execution to add a resource.
Inline CSS + inline SVG in report	Tailwind via CDN, external assets	Self-contained file. Works offline, attaches to PRs, no broken CDN risk.
ADC for local dev, WIF for CI	SA JSON keys everywhere	Avoid checking secrets into repos; WIF is GCP's modern recommendation.
Pivoted away from buildpack-deploy of `hello-flaky`	Fix Cloud Build IAM, use Dockerfile	Pragmatic. The tool's correctness is the deliverable, not the build pipeline. The synthetic test asserts the BREACHED case still works.
Conservative interpretation of "Valid Request" / "infrastructure 5xx"	Try to filter precisely	The metric doesn't expose what's needed; over-counting is documented and goes the safe direction.

What I'd build next

If this were day 2 of a real project, the order would be:

1. **Persist reports to GCS** for historical analysis. Trivial — one `bucket.blob(path).upload_from_string(html)` call in `report.py`.
2. **Add BigQuery and Cloud SQL** as service types. Both have well-defined SLAs and direct Cloud Monitoring metrics; the dispatch-table architecture absorbs them cleanly.
3. **Burn-rate alerts** — alert when the error budget is being consumed faster than expected, not just when it's already gone. The pattern is well- documented in Google's SRE workbook.
4. **A small UI** — `report.py` already produces HTML; serving it from a tiny Flask app behind IAP lets the team browse historical reports without spelunking through commits.

5. **Wire up real alerting —** Slack and PagerDuty.

Tools and references

- **Anthropic Claude** as a research and code-review partner throughout. I used it to parse SLA documents, rank metric-aggregation choices, draft the initial probe code structure, debug the Cloud Build failures (and decide when to pivot), and review the HTML rendering choices.
- **Official Google Cloud SLA pages —** single source of truth, fetched live rather than cited from memory:
  - https://cloud.google.com/run/sla (Cloud Run SLA, last modified Mar 25 2025)
  - https://cloud.google.com/storage/sla (Cloud Storage SLA, last modified Mar 4 2025)
- **Cloud Monitoring metric reference —** for the canonical metric names, labels, and resource types: https://cloud.google.com/monitoring/api/metrics_gcp
- **Cloud Operations SLI metrics guide —** for the request/response SLI patterns: https://cloud.google.com/stackdriver/docs/solutions/slo-monitoring/sli-metrics/req-resp-metrics

---

Closing notes

The two things I'd most want a reviewer to look at:

1. **The contract fidelity.** Cloud Run and Cloud Storage use different math because their SLAs say so. The tool keeps that asymmetry visible instead of papering over it.
2. **The honesty about modeling gaps.** Every place the measurement deviates from the literal contract is documented. A tool that *looks* right but silently overstates compliance is worse than one that's honest about the gaps.

The thing I'd most want to do differently if I started over: budget the Cloud Build IAM diagnosis up front so the live BREACHED demo could ship alongside the live COMPLIANT one. The synthetic test covers it, but a side-by-side live screenshot would be a stronger artifact.